Nesterov's Smooth Minimization of Non-Smooth Functions

Michał Szewczak Sebastian Trojan

June 22, 2026

Based on Yurii Nesterov, *Smooth minimization of non-smooth functions*, 2005.

Motivation

Many convex optimization problems are non-smooth because their objectives contain maxima, norms, or absolute values.

Classical subgradient methods for non-smooth convex optimization have a worst-case rate of order

$$O(1/\varepsilon^2).$$

Nesterov's smoothing method uses additional max-structure of the objective. It replaces the non-smooth term by a smooth approximation and then applies an accelerated first-order method. The resulting theoretical complexity is improved to

$$O(1/\varepsilon).$$

The main trade-off is controlled by the smoothing parameter: smaller μ gives a more accurate approximation, but a less smooth problem.

Structured problem class

The original problem is

$$f^* = \min_{x \in Q_1} f(x),$$

where

$$f(x) = \hat{f}(x) + \max_{u \in Q_2} \{\langle Ax, u \rangle - \hat{\phi}(u)\}.$$

- x is the primal variable.
- u is the auxiliary, or dual, variable.
- $\hat{f}(x)$ is the smooth part of the objective.
- $\hat{\phi}(u)$ is the part of the max-representation depending only on u.
- In the implemented examples, $\hat{f}(x) = 0$.

The non-smoothness comes from the maximization over u.

Smoothing the max-term

Choose a strongly convex prox-function d_2 on Q_2, normalized so that

$$\min_{u \in Q_2} d_2(u) = 0, \qquad D_2 = \max_{u \in Q_2} d_2(u).$$

Define the smoothed max-term

$$f_\mu(x) = \max_{u \in Q_2} \{\langle Ax, u \rangle - \hat{\phi}(u) - \mu d_2(u)\}.$$

The full smoothed objective is

$$\bar{f}_\mu(x) = \hat{f}(x) + f_\mu(x).$$

The approximation error satisfies

$$\bar{f}_\mu(x) \le f(x) \le \bar{f}_\mu(x) + \mu D_2.$$

Therefore, choosing

$$\mu = \frac{\varepsilon}{2D_2}$$

makes the smoothing error at most $\varepsilon/2$.

Gradient and smoothness trade-off

The smoothed dual response is

$$u_\mu(x) = \arg\max_{u \in Q_2} \{\langle Ax, u\rangle - \hat{\phi}(u) - \mu d_2(u)\}.$$

Then

$$\nabla f_\mu(x) = A^* u_\mu(x), \qquad \nabla \bar{f}_\mu(x) = \nabla \hat{f}(x) + A^* u_\mu(x).$$

The gradient of $\bar{f}_\mu$ is Lipschitz continuous with constant

$$L_\mu = M + \frac{\|A\|_{1,2}^2}{\mu \sigma_2}.$$

Here M is the Lipschitz constant of $\nabla \hat{f}$, and $\|A\|_{1,2}$ is the operator norm associated with the chosen primal and dual geometries.

Decreasing μ improves approximation accuracy, but increases L_μ.

Accelerated minimization

For a fixed smoothing parameter μ, the method minimizes

$$\min_{x \in Q_1} \bar{f}_\mu(x).$$

One accelerated iteration

$$\begin{aligned}
g_k &= \nabla \bar{f}_\mu(x_k) && \text{gradient,} \\
y_k &= \arg\min_{y \in Q_1} \left\{ \langle g_k, y - x_k \rangle + \frac{L_\mu}{2} \|y - x_k\|^2 \right\} && \text{local point,} \\
G_k &= \sum_{i=0}^{k} \alpha_i g_i, \qquad \alpha_i = \frac{i+1}{2} && \text{accumulated gradient,} \\
z_k &= \arg\min_{x \in Q_1} \left\{ \frac{L_\mu}{\sigma_1} d_1(x) + \langle G_k, x \rangle \right\} && \text{aggregate point,} \\
x_{k+1} &= \frac{2}{k+3} z_k + \frac{k+1}{k+3} y_k && \text{next search point.}
\end{aligned}$$

y_k uses the current gradient, while z_k uses all gradients accumulated in G_k.

Primal-dual certificate

The algorithm builds an averaged dual point

$$\hat{u} = \frac{\sum_{k=0}^{N} \alpha_k u_k}{\sum_{k=0}^{N} \alpha_k}, \qquad u_k = u_\mu(x_k), \qquad \hat{x} = y_N.$$

The dual lower-bound function is

$$\phi(u) = \min_{x \in Q_1} \{\hat{f}(x) + \langle Ax, u \rangle\} - \hat{\phi}(u).$$

By weak duality,

$$\phi(\hat{u}) \le f^* \le f(\hat{x}).$$

Therefore the primal-dual gap

$$f(\hat{x}) - \phi(\hat{u})$$

certifies the solution quality.

Theoretical iteration bound

Nesterov's estimate gives

$$f(\hat{x}) - \phi(\hat{u}) \leq \frac{4\|A\|_{1,2}}{N+1}\sqrt{\frac{D_1 D_2}{\sigma_1\sigma_2}} + \frac{4MD_1}{\sigma_1(N+1)^2}.$$

In the implemented examples, $\hat{f} = 0$, so $M = 0$. Hence

$$f(\hat{x}) - \phi(\hat{u}) \leq \frac{4\|A\|_{1,2}}{N+1}\sqrt{\frac{D_1 D_2}{\sigma_1\sigma_2}}.$$

To guarantee $f(\hat{x}) - \phi(\hat{u}) \leq \varepsilon$, it is sufficient to take

$$N_{\mathrm{pred}} + 1 = \left\lceil \frac{4\|A\|_{1,2}}{\varepsilon}\sqrt{\frac{D_1 D_2}{\sigma_1\sigma_2}} \right\rceil.$$

Thus the predicted complexity is

$$N = O(1/\varepsilon).$$

Problem classes

Matrix game: $$\min_{x\in\Delta_n}\max_{u\in\Delta_m} u^\top Ax$$

Continuous location: $$\min_{\|x\|_2\le r}\sum_{j=1}^{p} w_j\|x-c_j\|_2$$

Maximum of absolute values: $$\min_{\|x\|_2\le r}\max_{1\le j\le m}|a_j^\top x-b_j|$$

Sum of absolute values: $$\min_{\|x\|_2\le r}\sum_{j=1}^{m}|a_j^\top x-b_j|$$

Experimental plan

1. Reproduce Nesterov's matrix game experiment.
2. Apply the same scheme to the other three examples.
3. Test the empirical dependence on ε.
4. Compare standard smoothing with a changing-μ strategy.

In every experiment, convergence is measured by the certified condition

$$f(\hat{x}) - \phi(\hat{u}) \leq \varepsilon.$$

Experiment 1: matrix game reproduction

Reproduced problem:

$$\min_{x\in\Delta_n}\max_{u\in\Delta_m} u^\top Ax, \qquad A_{ij}\sim U[-1,1].$$

Here $A\in\mathbb{R}^{m\times n}$, n is the number of column strategies (dimension of x), and m is the number of row strategies (dimension of u).

Tested grid:

- $\varepsilon\in\{10^{-2},10^{-3},10^{-4}\}$,
- $m\in\{100,300,1000\}$,
- $n\in\{100,300,1000,3000,10000\}$, with the largest n omitted for $\varepsilon=10^{-4}$.

Each table cell reports iterations / percentage of the predicted iteration bound.

Experiment 1: matrix game results

$\varepsilon = 10^{-2}$

$m \backslash n$	100	300	1 000	3 000	10 000
100	765/42%	934/46%	1 108/49%	1 187/49%	1 281/49%
300	690/34%	910/40%	1 126/45%	1 268/47%	1 475/51%
1 000	761/34%	873/35%	1 086/39%	1 284/43%	1 457/46%

$\varepsilon = 10^{-3}$

$m \backslash n$	100	300	1 000	3 000	10 000
100	7 123/39%	8 426/41%	8 928/40%	10 191/42%	10 080/39%
300	7 337/36%	8 639/38%	10 154/40%	11 302/42%	12 022/41%
1 000	7 526/33%	8 828/35%	10 157/37%	11 595/39%	14 167/44%

$\varepsilon = 10^{-4}$

$m \backslash n$	100	300	1 000	3 000
100	68 505/37%	64 512/31%	77 136/34%	74 024/30%
300	60 932/30%	78 680/34%	83 844/33%	92 862/34%
1 000	70 778/31%	77 644/31%	93 346/34%	102 925/35%

Experiment 2: other problem classes

The matrix game experiment was extended to:

- Continuous location:
  $\min_{\|x\|_2 \le r} \sum_{j=1}^{p} w_j \|x - c_j\|_2, \quad c_j \sim \mathrm{Unif}(B_2(0, 0.8r)),\ w_j \sim U[0.5, 1.5],\ r = 1.$
  Grid: $p \in \{10, 50\}$, $d \in \{2, 5\}$ where $c_j, x \in \mathbb{R}^d$.
- Maximum of absolute values: $\min_{\|x\|_2 \le r} \max_{1 \le j \le m} |a_j^\top x - b_j|, \quad A_{ji}, b_j \sim U[-1, 1],\ r = 1.$
  Grid: $m \in \{100, 300\}$, $n \in \{50, 200\}$ where $a_j, x \in \mathbb{R}^n$.
- Sum of absolute values: $\min_{\|x\|_2 \le r} \sum_{j=1}^{m} |a_j^\top x - b_j|, \quad A_{ji}, b_j \sim U[-1, 1],\ r = 1.$
  Grid: $m \in \{100, 300\}$, $n \in \{50, 200\}$ where $a_j, x \in \mathbb{R}^n$.

Common accuracy levels:

$$\varepsilon \in \{10^{-2}, 10^{-3}, 10^{-4}\}.$$

Experiment 2: summary tables

Continuous location

ε	Avg. iter.	Max iter.	Avg. %	Time
10^{-4}	171 599	320 867	30.64%	31.8s
10^{-3}	20 525	31 653	43.74%	3.86s
10^{-2}	1 988	3 606	36.70%	0.35s

Maximum of absolute values

ε	Avg. iter.	Max iter.	Avg. %	Time
10^{-4}	171 486	249 861	39.81%	10.6s
10^{-3}	20 032	33 671	44.76%	1.26s
10^{-2}	2 593	3 788	56.06%	0.16s

Sum of absolute values

ε	Avg. iter.	Max iter.	Avg. %	Time
10^{-4}	1 198 404	2 711 368	29.45%	54.9s
10^{-3}	135 894	284 262	32.52%	6.43s
10^{-2}	12 088	27 750	30.43%	0.51s

Experiment 3: testing the $O(1/\varepsilon)$ behavior

To isolate the dependence on ε, five seeds were used; for each seed, one instance was generated and then kept fixed across all values of ε:

$$\varepsilon \in \{10^{-1}, 5 \cdot 10^{-2}, 10^{-2}, 5 \cdot 10^{-3}, 10^{-3}, 5 \cdot 10^{-4}, 10^{-4}\}.$$

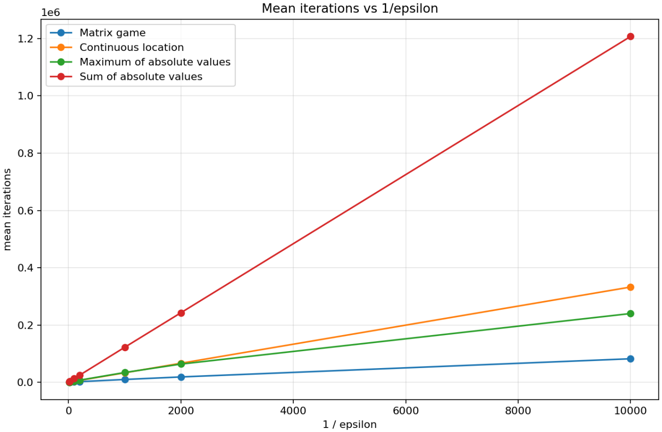
The theory predicts

$$N(\varepsilon) \approx C\frac{1}{\varepsilon}.$$

Therefore:

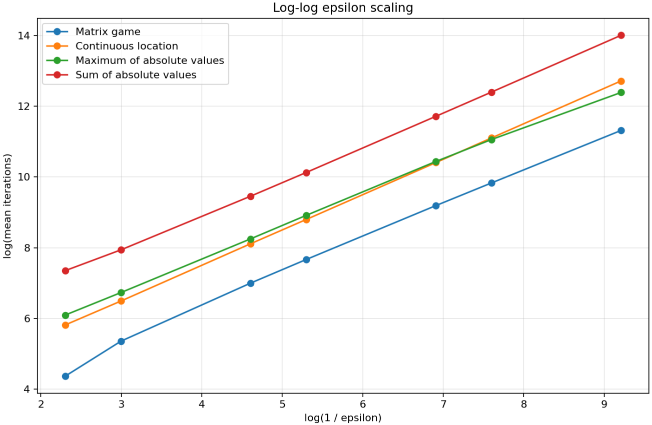
- iterations versus $1/\varepsilon$ should be approximately linear,
- the log-log fitted exponent $\hat{p}$ should be close to 1.

Experiment 3: iterations versus $1/\varepsilon$



The curves are close to linear; the difference between problems is mainly the constant factor.

Experiment 3: log-log scaling



Empirical power-law model:

$$N(\varepsilon) \approx C\varepsilon^{-p}.$$

Taking logarithms gives

$$\log N = \log C + p \log(1/\varepsilon).$$

Therefore $\hat{p}$ is the slope of the fitted line on the log-log plot. The theory predicts

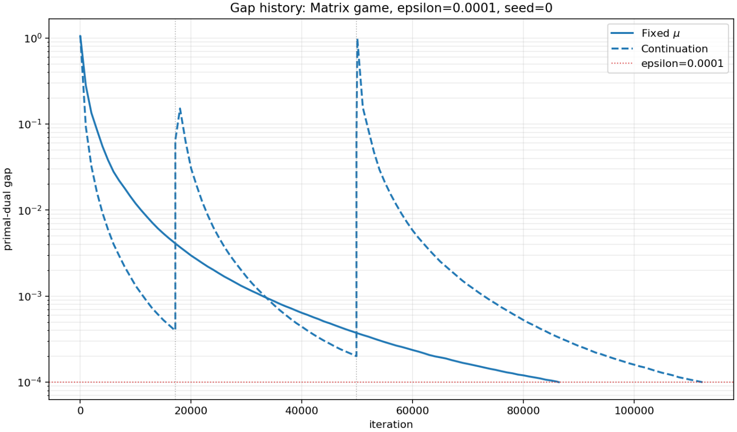
$$p = 1.$$

Problem	$\hat{p}$	SE
Matrix game	0.9897	0.0096
Continuous location	0.9986	0.0054
Max. abs. values	0.9208	0.0042
Sum abs. values	0.9649	0.0096

Experiment 4: changing the smoothing parameter

1. **Motivation**
   - Standard smoothing uses $\mu_\varepsilon = \varepsilon/(2D_2)$ from the beginning.
   - For small ε, this value is small, so the smoothed problem is sharper and harder.
2. **Method**
   - Start with a larger smoothing parameter and decrease it geometrically:
   $$\mu_0 = c\mu_\varepsilon, \qquad \mu_{t+1} = \rho\mu_t.$$
   - Move to the next stage when
   $$f(\hat{x}) - \phi(\hat{u}) \le c_{\text{stage}}\mu_t D_2.$$
3. **Our parameters**
   $$c = 8, \qquad \rho = 0.5, \qquad c_{\text{stage}} = 1, \qquad 8\mu_\varepsilon \to 4\mu_\varepsilon \to 2\mu_\varepsilon \to \mu_\varepsilon.$$

Experiment 4: gap history



After reducing μ, the smoothed objective changes and the primal-dual gap can jump upward. In most tested cases, continuation required more iterations.

Conclusions

- The implementation reached the required primal-dual gap on all reported instances.
- Matrix game results reproduce the qualitative behavior from Nesterov's paper.
- The empirical scaling is consistent with the theoretical $O(1/\varepsilon)$ complexity.
- Worst-case iteration bounds were conservative on random instances.
- Practical difficulty depends strongly on the problem structure.
- The tested changing-μ strategy was usually less efficient than the standard smoothing method.

Reference

Yurii Nesterov.
Smooth minimization of non-smooth functions.
Mathematical Programming, 103(1):127–152, 2005.